



Harness 잘 사용하기

챗봇과 싸우지 않고, 에이전트가 일할 환경을 설계하는 방법

게임플랫폼 1팀 라승수

시작하며

01

챗봇과 말씨름

맥락 충돌과 반복 설명

02

도구들의 수렵

자동완성·채팅 UI·터미널 에이전트

03

Harness

규칙·컨텍스트·권한·검증·상태 기록

개발자의 핵심 역량: AI에게 할 말 → AI가 안전하게 일할 환경

OPENING

전체 목차

01	코딩은 어디로 가고 있는가	개발자의 숙련 이동
02	왜 Claude Code인가	AI 코딩 도구의 구조 수렴
03	AI 시대의 개발 방법론	TDD·SDD·Spec-first 재부상
04	Harness	Prompt 이후의 에이전트 운영 구조
05	한계와 실패 패턴	긴 작업·보안·신뢰 문제
06	멀티 에이전트	역할과 컨텍스트 분리
07	워크플로우와 도구	명령어·게이트·워크트리·이슈
08	제작 과정	발표 제작 파이프라인 해부
09	다음에 해야 할 일	개인과 팀의 첫 변화

CHAPTER 01

코딩은 사라지는가

기계어 → 어셈블리

기계어

```
10111000 00000001 00000000 00000000 00000000
```

어셈블리

```
MOV EAX, 1
```



컴파일러가 만든 코드가 사람보다 효율적일 리 없다!

어셈블리 → C/Pascal

어셈블리

```
section .data
    msg db "Hello World"

section .text
    global _start

_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 12
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```



C/Pascal

```
#include <stdio.h>

int main() {
    printf("Hello\n");
    return 0;
}
```

진짜 프로그래머는 파스칼 같은 걸 쓰지 않는다!

C → Java

C

```
#include <stdio.h>
#include <stdlib.h>

...

int main() {
    Point* p = (Point*)malloc(sizeof(Point));
    p->x = 10;
    p->y = 20;
    printf(
        "Point: %d, %d\n",
        p->x,
        p->y
    );
    free(p);
    return 0;
}
```

Java

```
Point p = new Point(10, 20);
System.out.println(
    "Point: " + p.x + ", " + p.y
);
p = null;
```



메모리를 직접 관리하지 않으면 진짜 개발자가 아니다!

Java → Python

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```



Python

```
print("Hello, World!")
```

장난감 언어로 무슨 개발을 하냐!

AI 개발

자연어

이 함수를 리팩토링하고 테스트 코드를 작성해줘.

자연어로 시키는 건 진짜 개발이 아니다!

직접 하던 일의 감소

시대	전환	위임한 것
1950s	기계어 → 어셈블리	이진 코드를 직접 작성하는 부담
1960s	어셈블리 → C	레지스터와 명령어 최적화를 컴파일러에 위임
1990s	C/C++ → Java	메모리 해제를 가비지 컬렉터에 위임
2000s	언어 → 프레임워크	반복되는 보일러플레이트를 프레임워크에 위임
2010s	온프레미스 → 클라우드	서버 운영과 확장을 클라우드에 위임
2020s	직접 코딩 → AI 에이전트	코드 작성의 상당 부분과 작업 환경 설계

직접하는 일을 줄고, 설계하는 일은 늘어난다

숫자로 보는 변화

25%

YC W25

코드베이스 95% 이상 AI

100만 줄

OpenAI Codex 팀

수동 코드 한 줄 없음

3개월+

Meta 엔지니어

직접 코드 타이핑 없음

문서가 코드

이름은 다르지만 역할은 동일



Claude Code

CLAUDE.md



Cursor

Cursor Rules



GitHub Copilot

**copilot-
instructions.md**



Codex

AGENTS.md

개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

건축가

벽돌 쌓기 대신 설계도 작성

정확한 설계도 → 튼튼한 건물

오케스트라 지휘자

직접 연주 대신 전체 조율

CLAUDE.md = 총보

그래도 기초가 중요하다

AI가 더 많이 해줄수록 기초 지식을 가진 사람의 경쟁력 상승

변하는 것

직접 코드를 타이핑하는 비중

줄어드는 것은 타이핑의 비중

변하지 않는 것

시스템을 이해하는 능력

안전성 · 유지보수성 · 기존 시스템 적합성
인증과 권한 · 동시성 · 데이터베이스 스키마 · 캐싱

CHAPTER 02

왜 Claude Code인가

에이전틱 코딩의 실제 성과

2주

CTO 추정 4~8개월

프로젝트 완료

카카오 AI 팀 내부 공유 사례

1~2일

신규 개발자 온보딩

기존 수주~수개월에서 단축

첫날부터 의미 있는 기여

**불가능하던 작업
실현**

기술 부채 해결

즉각 피드백과 작업 실현

문서화된 작업 환경

1막: Copilot과 ChatGPT, 프롬프트의 시대

2022~2024

GitHub Copilot

현재 파일 기반 다음 줄 제안

현재 파일

암묵적 프롬프트

자동완성



ChatGPT

자연어 지시 기반 코드 생성

프롬프트 길이

역할 부여

단계별 지시

자연어 지시문이 새로운 프로그래밍 인터페이스처럼 보이던 시기

Chain-of-Thought

중간 추론 단계를 쓰게 한다

답만 내던 모델을 reasoning
path가 있는 모델처럼 다루기
시작

문제

$$5\text{개} + 2\text{칸} \times 3\text{개}$$

중간 추론

$$2\text{칸} \times 3\text{개} = 6\text{개}$$

답

$$5\text{개} + 6\text{개} = 11\text{개}$$

reasoning path

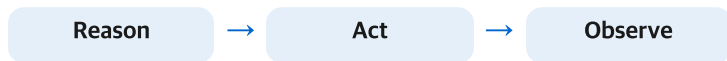
답만 내던 모델을 reasoning
path가 있는 모델처럼 다루기
시작

ReAct / Tree-of-Thought

02

ReAct

추론과 행동을 번갈아 수행



검색과 도구 호출이 루프 안으로 진입

03

Tree-of-Thought

여러 추론 경로를 동시에 탐색



하나의 답이 아니라 후보 경로를 비교

Self-Refine / Reflexion

Self-Refine

자기 출력을 다시 비판하고 수정

초안



비판



수정

초안 이후의 개선 루프

Reflexion

자기 출력을 다시 비판하고 수정

실패



기억



재시도

실패 이후의 피드백 루프

네 가지 에이전틱 패턴

Reflection

한 번 낸 답을 다시 보고 고침

Generate → Critique → Revise

자기 수정 루프

Tool Use

검색·계산·실행을 바깥 도구에 맡김

Model → Search/API → Result

모델 바깥 세계 연결

모델 바깥 구조가
성능을 좌우

Planning

목표를 단계로 쪼갬

Goal → Plan → Steps

긴 작업 추진

Multi-Agent Collaboration

역할을 나눠 다른 관점으로 봄

Writer → Reviewer → Merge

생성과 검증 분리

프롬프트 시대의 벽

모델은 보지 못한 것을 알 수 없음

관련 파일 부재

기존 유틸리티 인식 불가

기존 설계 부재

설계 결정 활용 불가

팀 컨벤션 부재

내부 규칙 활용 불가

문제는 지시문이 아니라 모델이 소비하는 정보

2막: Cursor와 컨텍스트의 시대

초기 Copilot

현재 파일 중심
검색 없음 또는 제한
한 줄 또는 한 블록



Cursor 계열 도구

전체 코드베이스
RAG · AST · 시맨틱 검색
멀티파일 수정과 Tools

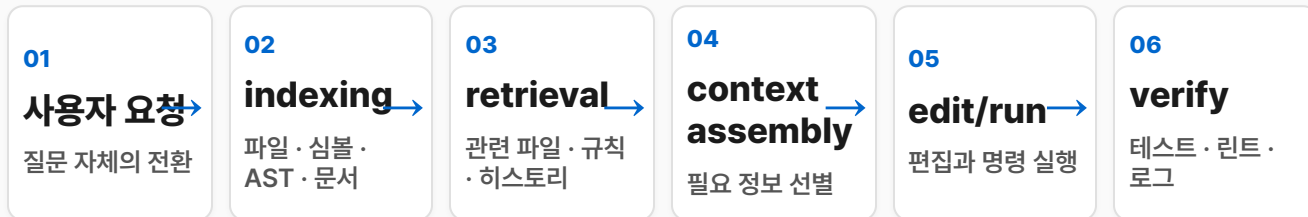
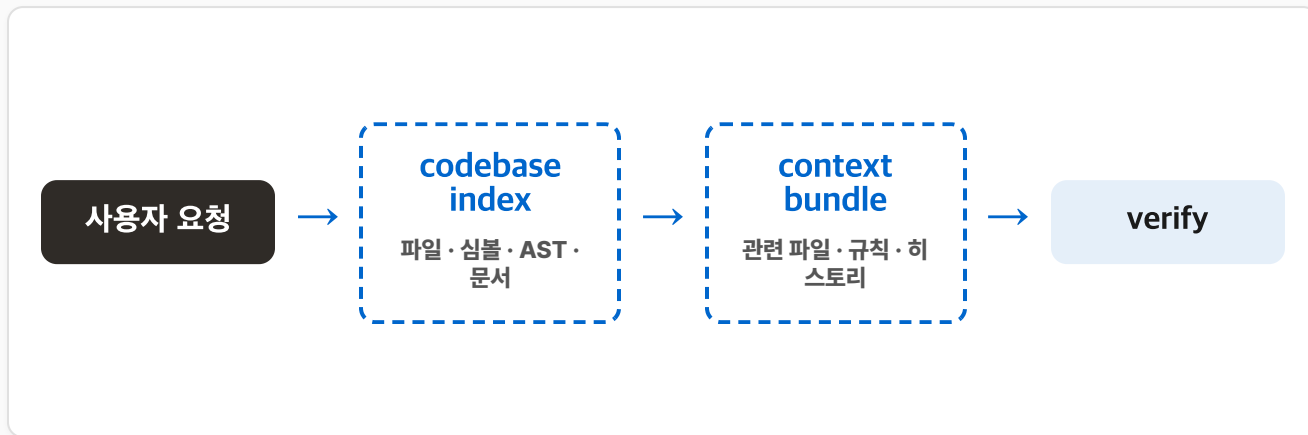
채팅

Composer

Agent Mode

Context Engineering

Cursor 아키텍처



TOOLS

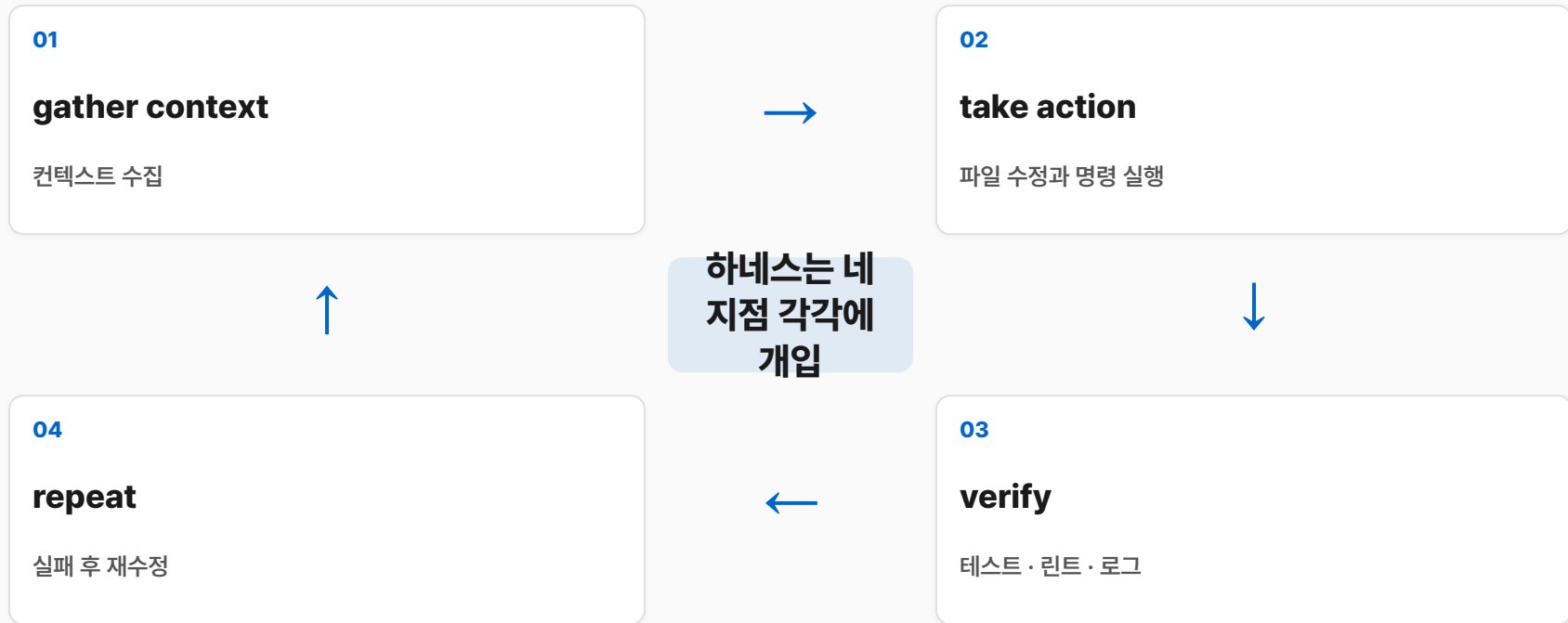
채팅

Composer

Agent Mode

명령 실행

컨텍스트 시대의 벽



3막: 하네스의 시대

작업 환경 전체를 품기 시작한 코딩 도구

터미널 네이티브

파일 읽기 · 셸 실행 · 테스트 확인

구성 요소의 묶음

독립 기능이 실행 환경으로 결합

내 환경에 맞는 도구

플러그인 · 스킬 · 노하우 · 교육자료

Plan Mode

승인 체계

CLAUDE.md

Skills

Hooks

MCP

Plugins

Subagents

시대의 흐름

2022~2024

Prompt Engineering

어떤 말을 해야 하나

2025 중반~

Context Engineering

어떤 정보를 넣어야 하나

2026~

Harness Engineering

어떤 시스템을 만들어야 하나

Agent = Model + Harness

Prompt $\subset$ Context $\subset$ Harness

시대	핵심 질문	주안점
Prompt Engineering	어떤 말을 해야 하나	자연어 지시문과 기법
Context Engineering	어떤 정보를 넣어야 하나	코드베이스와 구조화 정보
Harness Engineering	어떤 시스템을 만들어야 하나	권한 · 피드백 루프 · 검증